ENGS 147: Mechatronics
Lab 0: Arduino Stack Setup

Spring 2026

Objective: In this lab, you will become familiar with the Arduino microcontroller system and set up a software development environment that you will use in all of the labs for this course.

Lab kit contents: Open your lab kit, and confirm that you have each of the following items:

- (1) Pair of wire strippers
- (1) Small electronics screwdriver
- (1) Handheld multimeter
- (1) Tenergy NiMH battery
- (1) Tenergy RC battery charger
- (1) Arduino Due board
- (1) Robogaia 3 axis encoder shield (Note: these are no longer in production, so please be careful!)
- (1) Arduino motor shield
- (1) Arduino 9 axis motion shield
- (1) USB programming cable (A to micro B or C to micro B – see instructor if you need a different cable)
- (1) Set of encoder cables
- (1) Tamiya connector cable (connects battery to motor shield)
- (1) Pair of 18 gauge wires (red and black, connect motor shield to motor)
- (1) Pancake motor board

1. Plug in your battery to ensure that it is charged. Leave your charger on the 1C setting.
2. **Set up your Arduino development environment**
   - **Recommended option: Install Arduino IDE (integrated development environment)**
     - Download and install Arduino IDE 2.3.8 (or newer) from https://www.arduino.cc/en/software
     - Launch the Arduino IDE
     - Find your sketchbook location (PC: File > Preferences; macOS: ArduinoIDE > Settings)
     - Open your sketchbook folder in Windows Explorer (PC) or Finder (macOS)
     - Create a folder called `libraries` in your sketchbook folder if it doesn't already exist
     - Copy the course libraries from https://dartgo.org/147lib to the `libraries` folder
     - Close and reopen the Arduino IDE (this reloads your libraries and examples)
   - **Alternative options (use at your own risk – or benefit!)**
     - VSCode with PlatformIO
       - Provides code completion and other features not available in Arduino IDE
       - Instructions available from various sources online
       - Course staff will provide limited support
     - Arduino Cloud: https://cloud.arduino.cc/
       - Code online via your browser
       - Requires installing Arduino Create Agent to program the Due
       - Library management may be somewhat more challenging
       - Integrated serial monitor works better, but its output may require additional parsing

3. **Test basic Arduino functionality**
   - Determine the pin mapping for the amber "L" LED on the Arduino Due board using the pin mapping reference: https://docs.arduino.cc/retired/hacking/hardware/PinMappingSAM3X/
   - Open Arduino IDE and add the following sketch (or download `blink_147.ino` from https://dartgo.org/147examples):

```
#define LED_PIN xx   // replace xx with Due digital pin number for LED

void setup(){
  // initialize the serial port using a baud rate of 115,200 bits per second
  Serial.begin(115200);
  Serial.println("Hello, world!");

  // configure the digital pin attached to the LED as an output
  pinMode(LED_PIN,1);
}

void loop() {
  // declare and initialize variables
  uint8_t led_state = 0;
  uint32_t i = 0;

  // infinite loop
  while(1){

    // print integer to serial port
    Serial.println(i);
    ++i;

    // blink LED
    digitalWrite(LED_PIN,led_state);
    led_state = ~led_state;

    // wait 0.5 sec
    // NOTE: this is *NOT* a good way to enforce loop timing!
    delay(500);
  }
}
```

   - Save your sketch.
   - If you are unfamiliar with the function of any of these lines of code, please review with the instructor, a TA, or a classmate.
   - Connect your Arduino Due to your laptop with a USB cable. Use the centermost micro USB port on the Arduino Due – this is its programming port.

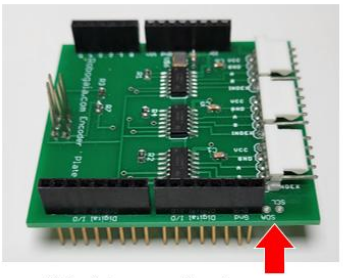
- In the Arduino IDE, expand the "Select Board" dropdown menu and select the "Arduino Due (Programming Port)" entry. Install the Arduino SAM Boards package if you are prompted to do so. On macOS you may be prompted to install command line developer tools as well (this may take some time).
- Click the Upload arrow to compile and upload your sketch to the Arduino Due.
- The Arduino IDE comes with a serial monitor that allows you to view text printed by your embedded programs. Historically this terminal did not allow you to easily save data retrieved. Recent updates added a "copy" option which you can use to copy/paste data into a new text or .csv file if you wish. If you would prefer direct save-to-file functionality, we recommend a free, cross-platform program called CoolTerm which you can download here: https://freeware.the-meiers.org/#CoolTerm. You may use a different terminal program if you wish (e.g. RealTerm on Windows or Minicom on Linux), but the course staff are best prepared to help debug issues with Arduino IDE and CoolTerm.
- Testing with Serial Monitor in Arduino IDE:
    - Make sure you have compiled and uploaded the `blink_147.ino` sketch to your board.
    - Open the Arduino serial monitor from within the IDE (Tools > Serial Monitor)
    - Change the Serial Monitor baud rate to 115,200 bits/sec.
    - You should now see numbers appearing on your screen twice per second.
    - Click the "Clear Output" button in the Serial Monitor toolbar.
    - Wait several seconds for additional numbers to appear in your console.
    - Click the "Copy Output" button in the Serial Monitor toolbar.
    - Open a text editor on your laptop and paste the copied output into a new document. You need not save this document now, but may wish to refer back to these steps when you begin capturing data from your Arduino that you wish to save for analysis.
- Testing with CoolTerm (if you choose to use it):
    - If you opened the serial monitor in the Arduino IDE, close it now.
    - Open CoolTerm. If you are using a Mac, you may need to right click and select "Open" rather than double clicking the CoolTerm icon for the first launch. From the quick menu at the lower left of the window, change the baud rate to 115200, and select the correct port (on macOS it may be called something like "usbmodem"), then press the "Connect" button. You should see "Hello, world!" and a 2Hz counter displayed. Press the "disconnect" button.
    - Note: You will need to disconnect CoolTerm (but need not close its window) to program again from the Arduino IDE. If you do not like repeatedly opening and closing the serial port in CoolTerm, you can program the Due via the programming port and print serial output to Due's other USB port. In this case, connect a second cable from the outboard micro USB port to another

USB port on your laptop. In you sketch, change all instances of Serial to SerialUSB (e.g. use: SerialUSB.begin(115200)).

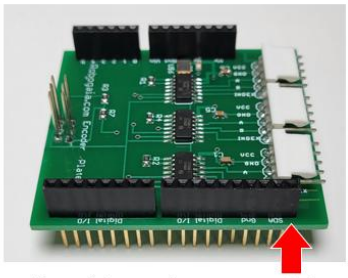
- Finally, test capturing the serial output of your Arduino program directly to a file. In CoolTerm, select Connection > Capture to Text/Binary File > Start. Choose a file location and press "Save." Next, press the "Connect" button. Upon connection the Arduino will reset and restart its program. The status bar will indicate that data are being captured. After a few seconds, select Connection > Capture to Text/Binary File > Stop. Open the file that was saved in a text editor to confirm your capture. In the future your Arduino programs will print lines of comma-separated numerical values, and you will save your capture as a CSV file for import into MATLAB (or Excel, etc.) for analysis.

4. Install your shields

- Find your encoder shield. This board contains three LS7366R-S quadrature counter chips that interface with standard optical encoders. The encoder shield itself is no longer manufactured, however you can find the schematic here. There are two types of encoder board in our lab kits: an older version that is missing a pass-through connection for the I^2C bus, and a newer version with a full set of connectors. Using the photos below, determine which version you have in your kit.

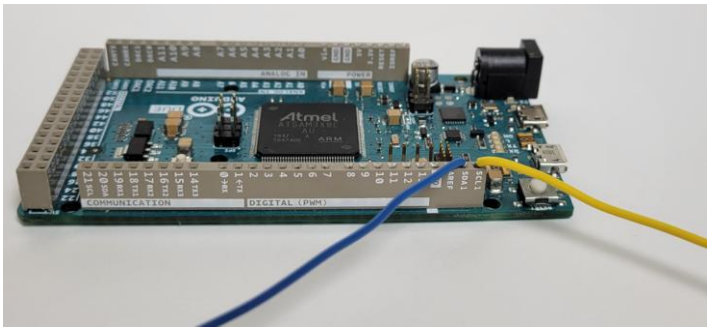


Old style – requires jumpers

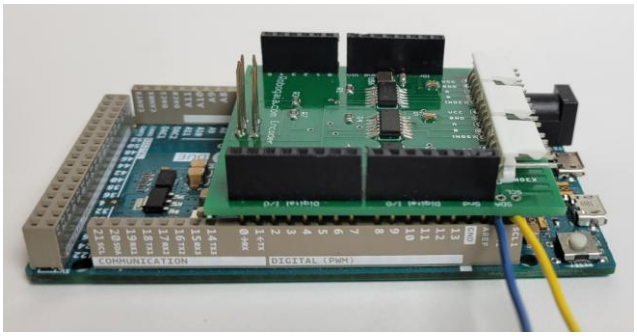


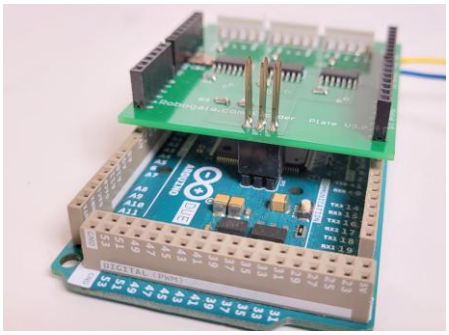
New style – no jumpers needed

If you are using the older style encoder board, you will need to add solid hookup wire jumpers to your stack. Carefully strip two jumper wires and insert one into each of the SCL1 and SDA1 header connections on your Arduino Due board as shown below.

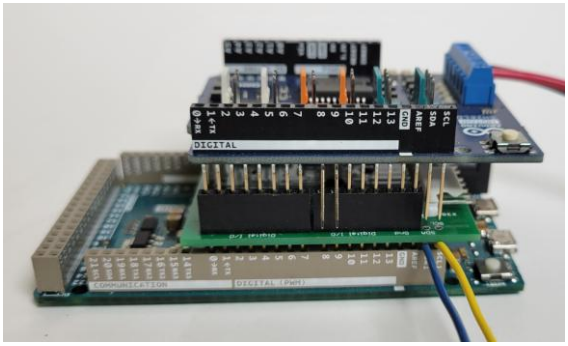


- Next attach your encoder shield to your Due board, being careful to avoid bending pins. Note that the six-position SPI (serial peripheral interface) header in the center of the Due board will connect to a receptacle on the encoder board.





- Next locate your motor shield. Pins 8 and 9 of the motor shield (labelled `BRAKE DISABLE` underneath) conflict with encoder shield signals, and must therefore not be connected. Slightly bend these pins outward and connect your motor shield to the encoder shield as shown below.



- Finally, locate your 9-axis motion shield and attach it to the motor shield as shown below. If you are using I^2C jumper wires, connect them to the SCL and SDA receptacles on the 9-axis motion shield.